

- 1) Assembly
 - 2) Process Management
 - 3) Memory Management
 - 4) File Management
 - 5) Device Management
-

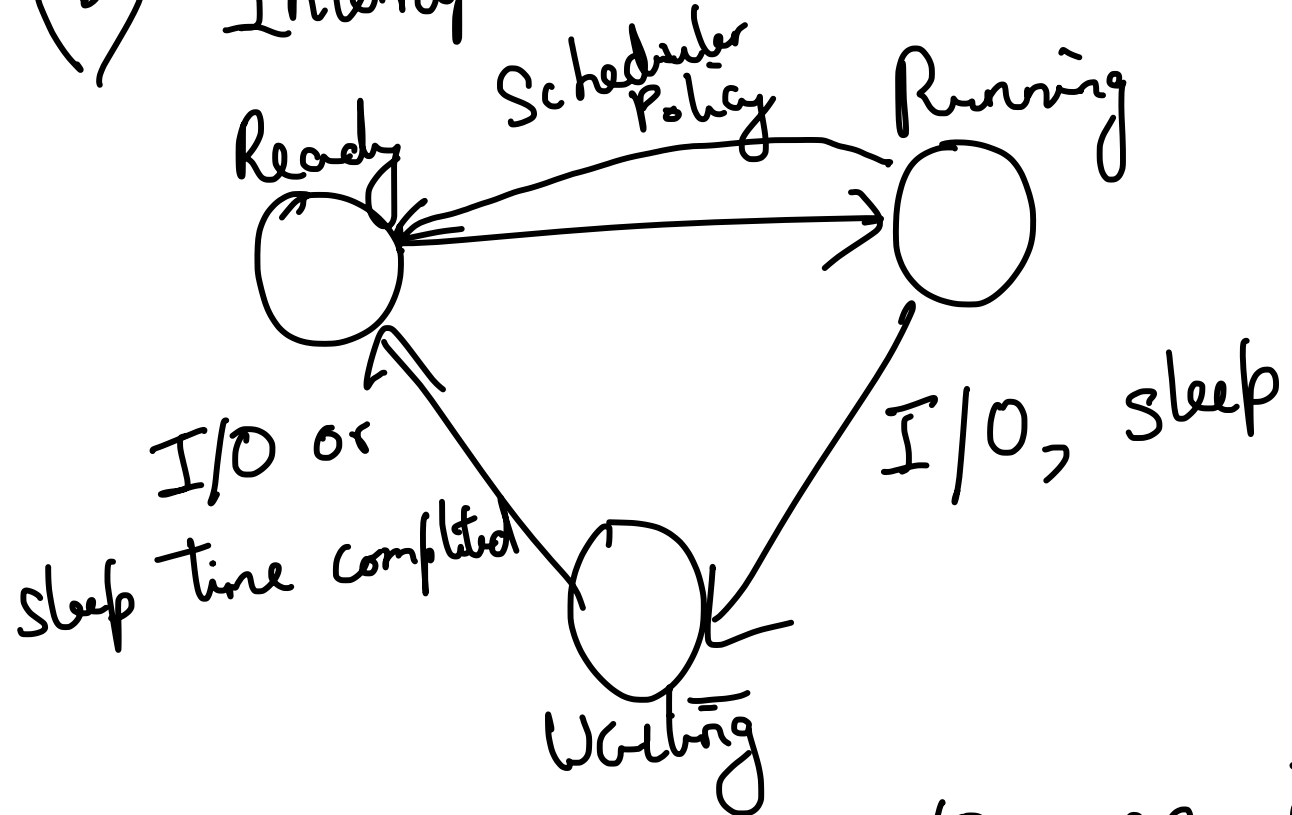
Device Management

How is I/O performed?

(a) Polling → Method of doing I/O where the processor sends a request to the device and waits until the device sends a

response. Polling is usually not feasible because of slower I/O compared to processor.

(b) Interrupt-driven I/O → Whenever a



process requires I/O, an interrupt is raised which sends the process to waiting state and activates the process scheduler. When I/O is completed, the device sends another signal, and then the process is sent to the ready state.

What are the disadvantages and ways of mitigating them?

② Every interrupt raised comes with an overhead,

→ Network card of a web server

→ Too many context switches are possible, making it unprofitable.

→ Interrupt coalescing → Joining multiple requests into one.

→ Reading from/writing to secondary storage

(Can be about very large data, but is more predictable in nature)

③ Direct Memory Access (DMA): → Interrupt is needed only to initiate the request. Since the data need to read/write is predictable, so processor's instruction is not needed.

at each step. Instead, the control is given to the bus / I/O device to complete the request.

(1) Do we need abstractions for devices?

→ To facilitate easy interaction with programs

→ Need multiple layers because of the diversity in the type of devices.

→ One abstraction is the classification of character and block devices

mouse, keyboard,
printer



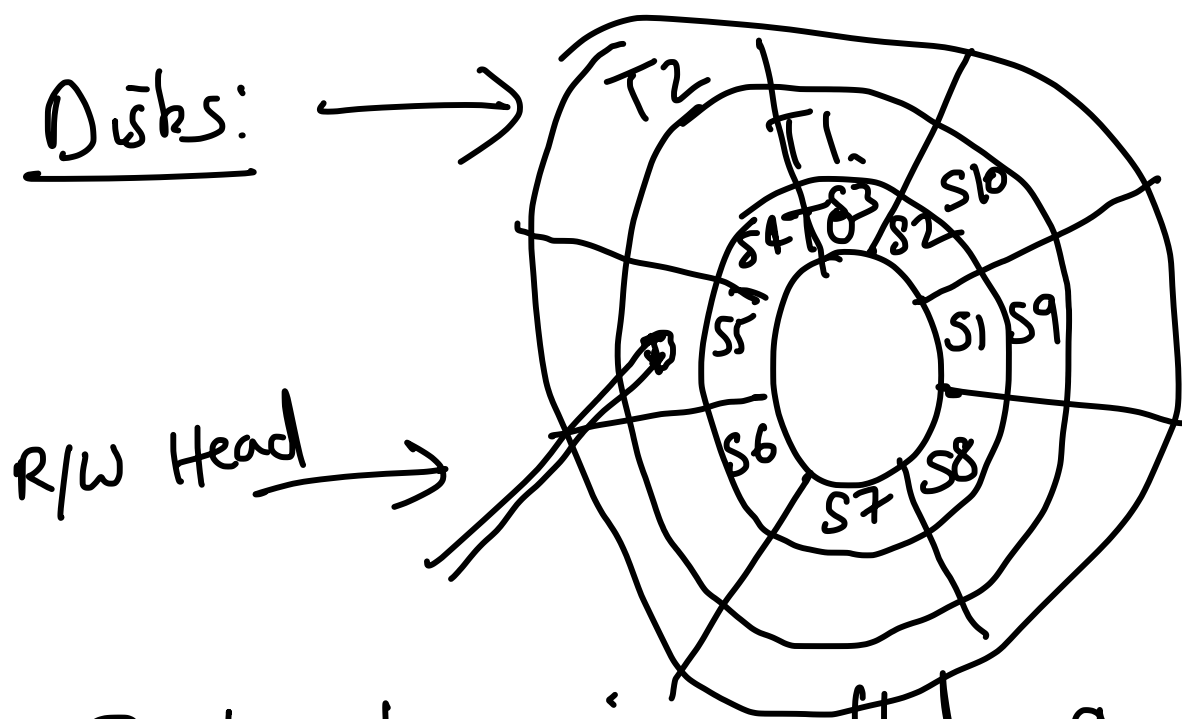
data has to be
either read in sequence
or written in sequence

Secondary Storage



data can be
-accessed at random

Secondary storage: $\rightarrow$ Disks & SSDs



Each layer is called a track, and tracks consist of sectors. Each sector consists of 512 ^{bytes} ~~words~~ of data.

The read/write head has to physically move over the sector before reading or writing can be performed.

[Reliability and life time of a disk $\rightarrow$ OS cannot do much to improve]

$\rightarrow$ Read/write speeds depend on the distance between the write sector and the read/

write head.

→ Splitting of files might be needed if they are large in size.

→ Fragmentation can make reading/writing of files much slower.

(Defragmentation is a way of reorganizing the files so that each file content is kept in contiguous locations on the disk).

Time to access data on a disk
= Seek Time + Rotation Time + Transfer Time

(Time taken for r/w head to reach the right track)

(time taken for r/w head to read the right sector within the track)

What can device management do to optimize disk I/O?

→ Organize the files in such a way that

movement of r/w head is minimized.

→ Wait for a number of requests, and then rearrange them/schedule them to reduce the movement of r/w head.

[Requires I/O scheduler, which can have built-in intelligence]

→ Disk cache → Store the data in the main memory, earmark it as being written on the disk and return to the program

`fsync()` system call → forces the writes to actually complete on the disk.

Sync Command on bash

`open` → flag called `O_SYNC`

⇓
Comes with a performance penalty as the write system call becomes slower.

Files can also be organized in such a way that the requirement of fragmenting is minimized.

— File system decides how the files are actually organized.

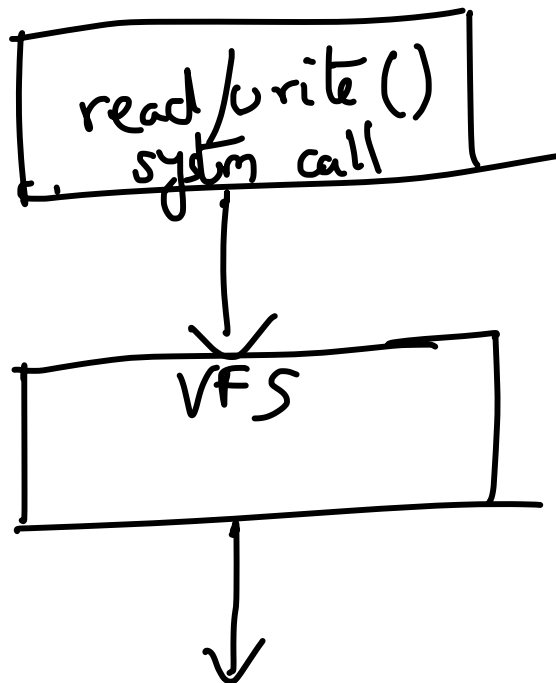
Windows → NTFS

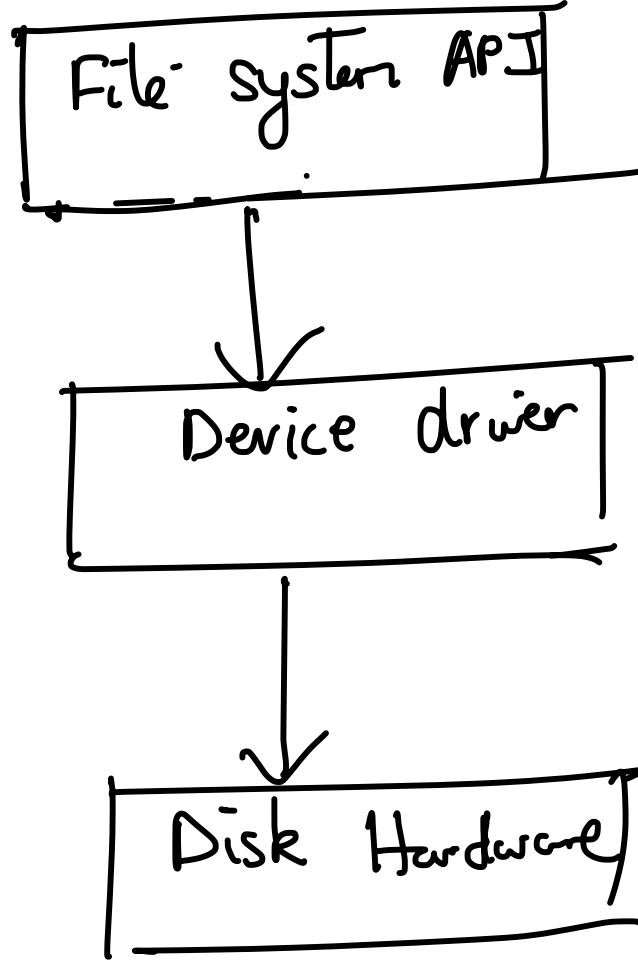
Linux → Ext4, Btrfs, ZFS

Organize the data in different ways. These data organization techniques influence the data access times and fragmentation.

How can a program handle all these different file systems?

↔ Linux provides the virtual filesystem (VFS) as an abstraction layer.





User programs only need to understand the type of operations supported by virtual file system.